

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #8 - Semana 9 – Sivarama - Capitulo 14
Instrucciones aritméticas

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 23/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 291–316.

Abstract—State flags play a crucial role in monitoring arithmetic and logical operations in assembly language. This paper discusses key flags such as the Zero Flag (ZF), Carry Flag (CF), Overflow Flag (OF), Sign Flag (SF), Auxiliary Flag (AF), and Parity Flag (PF), and their significance in conditional branching and arithmetic operations. Examples are provided for using ZF in conditional jumps and CF in unsigned arithmetic operations. The paper also covers instructions for multiplication and division, highlighting the complexities involved, particularly for signed versus unsigned numbers. Furthermore, it explains the necessity of sign-extension instructions, ensuring accurate results during division operations.

I. Las banderas de estado

Las banderas de estado son esenciales para el monitoreo de las operaciones aritméticas y lógicas en ensamblador. Estas banderas incluyen:

1. **Zero Flag (ZF):** Indica si el resultado de la última operación aritmética es cero. Si es cero, ZF = 1, de lo contrario, ZF = 0. Ejemplo:

```
mov AL, 0FH
add AL, 0F1H ; AL = 00H, ZF = 1
```

2. **Carry Flag (CF):** Se activa cuando el resultado de una operación aritmética excede la capacidad del registro de destino (desbordamiento) o si es demasiado pequeño para representarse (subdesbordamiento). Ejemplo:

```
mov AL, 0FFH
add AL, 01H ; AL = 00H, CF = 1
```

3. **Overflow Flag (OF):** Indica si el resultado de una operación aritmética de números con signo ha superado el rango representable.
4. **Sign Flag (SF):** Indica si el resultado de una operación aritmética es negativo.
5. **Auxiliary Flag (AF):** Se utiliza para operaciones de ajuste de BCD (Binary Coded Decimal) al indicar desbordamientos entre los bits de menor orden.
6. **Parity Flag (PF):** Indica si el número de bits con valor 1 en el resultado es par.

II. Uso de la bandera de cero

La bandera de cero se utiliza frecuentemente en instrucciones condicionales. Por ejemplo, las instrucciones de salto condicional **jz** (salta si ZF = 1) y **jnz** (salta si ZF = 0) permiten cambiar el flujo del programa basándose en si el resultado de una operación fue cero o no.

Ejemplo de comparación para verificar igualdad usando la instrucción `cmp`:

```
cmp EAX, EBX ; ZF = 1 si EAX es igual a EBX
jz igual      ; salta a la etiqueta 'igual' si ZF = 1
```

III. Uso de la bandera de acarreo

La bandera de acarreo es fundamental para operaciones con números sin signo, donde se utiliza para manejar desbordamientos. Se puede propagar un acarreo en operaciones de suma o resta de múltiples palabras. Ejemplo de suma de dos números de 64 bits usando registros de 32 bits:

```
add EAX, ECX ; suma las partes bajas
adc EBX, EDX ; suma las partes altas con acarreo
```

La instrucción `adc` añade el valor del acarreo (si existe) a la operación actual.

IV. Instrucciones de multiplicación y división

Las operaciones de multiplicación y división en ensamblador requieren instrucciones distintas para números con y sin signo:

1. **Multiplicación (MUL e IMUL):**
 - MUL: Multiplica sin signo.
 - IMUL: Multiplica con signo.
2. **División (DIV e IDIV):**
 - DIV: Divide sin signo.
 - IDIV: Divide con signo.

Estas operaciones suelen implicar un solo operando explícito, mientras que el otro operando se asume que está en un registro predefinido. Por ejemplo, en la

multiplicación MUL, el operando multiplicado por el valor en el registro AX para 16 bits o EAX para 32 bits.

V. Ejemplos de código y explicación

1. **Incremento sin afectar CF:** La instrucción INC incrementa el valor de un registro sin modificar la bandera de acarreo, lo que la hace útil para bucles. Ejemplo:

```
mov EAX, 0
inc EAX ; EAX = 1, CF no se modifica
```

2. **Suma de números de múltiples palabras:** Para sumar dos números de 64 bits almacenados en dos pares de registros (EAX y ECX), se utiliza la siguiente secuencia de instrucciones:

```
add EAX, ECX ; suma las partes bajas
adc EBX, EDX ; suma las partes altas con acarreo
```

3. **Bucle con loop y dec:** Un bucle puede controlarse decrementando un registro y verificando la bandera de cero:

```
mov ECX, 10 ; contador del bucle
loop_label:
; código dentro del bucle
dec ECX
jnz loop_label ; repite el bucle hasta que ECX sea 0
```

	Code	AL	CF	ZF	SF	OF	PF
Example 1	mov AL, -5						
	sub AL, 123	80H	0	0	1	0	0
Example 2	mov AL, -5						
	sub AL, 124	7FH	0	0	0	1	0
Example 3	mov AL, -5						
	add AL, 132	7FH	1	0	0	1	0
	add AL, 1	80H	0	0	1	1	0
Example 4	sub AL, AL	00H	0	1	0	0	1
Example 5	mov AL, 127						
	add AL, 129	00H	1	1	0	0	1

Figura 1. Ejemplos del efecto en las banderas.

VI. Instrucciones aritméticas

La arquitectura IA-32 soporta diversas instrucciones aritméticas, que incluyen las operaciones de **suma** y **resta** (vistos previamente), además de otras importantes como multiplicación y división.

1. **Suma:** add, adc, inc

2. **Resta:** sub, sbb, dec, neg, cmp
3. **Multiplicación:** mul, imul
4. **División:** div, idiv

Otras instrucciones relacionadas con números decimales y BCD se abordan en otros capítulos.

VII. Instrucciones de multiplicación

La multiplicación en IA-32 es más compleja que la suma y la resta por dos razones:

1. **Resultado de doble longitud:** Multiplicar dos valores de n bits produce un resultado de $2n$ bits. Por ejemplo, la multiplicación de dos números de 8 bits genera un resultado de 16 bits, mientras que dos números de 32 bits generan un resultado de 64 bits.

Ejemplo (sin signo, 8 bits):

$$11111111 \times 11111111 = 1111110111111111 \quad (255 \times 255 = 65025)$$

2. **Números con y sin signo:** Los números con y sin signo producen diferentes patrones de bits. Ejemplo:
 - **Sin signo:** 11111111×11111111 produce 1111110111111111 ($255 \times 255 = 65025$).
 - **Con signo:** 11111111×11111111 produce 0000000000000001 ($-1 \times -1 = 1$).

Para abordar estas diferencias, existen dos instrucciones de multiplicación:

- **mul** para números sin signo.
- **imul** para números con signo.

Instrucción mul: La instrucción mul se utiliza para multiplicar números sin signo. El operando fuente puede ser un registro o una posición en memoria, y la instrucción asume que el otro operando está en el acumulador (registro AL, AX, o EAX). Dependiendo del tamaño del operando, los resultados se almacenan en registros como sigue:

- Para un operando de 8 bits: el resultado de 16 bits se almacena en AX.
- Para un operando de 16 bits: el resultado de 32 bits se almacena en DX:AX.
- Para un operando de 32 bits: el resultado de 64 bits se almacena en EDX:EAX.

El mul afecta las banderas de acarreo (CF) y desbordamiento (OF), que se activan si la mitad superior del resultado no es cero.

Ejemplo de uso:

```
mov AL, 10
mov DL, 25
mul DL
```

El resultado será 250, que cabe en AX, por lo que las banderas CF y OF se mantienen limpias.

Instrucción imul: La instrucción imul se utiliza para la multiplicación de números con signo. Su comportamiento es similar a mul, pero las banderas CF y OF se activan si la mitad superior del resultado no es la extensión del signo de la mitad inferior.

Ejemplo:

```
mov DL, 0FFH ; DL = -1
mov AL, 42H  ; AL = 66
imul DL      ; Resultado: AX = -66 (1111111110111110)
```

En este caso, las banderas CF y OF no se activan, ya que la parte superior del resultado es la extensión del signo.

VIII. Instrucciones de división

La división también es más compleja que la suma y la resta, ya que genera dos resultados: **cociente** y **residuo**. Además, hay dos versiones de la instrucción de división:

- **div:** para números sin signo.
- **idiv:** para números con signo.

En ambos casos, el dividendo tiene el doble de longitud que el divisor, y los resultados se almacenan como sigue:

- Para divisores de 8 bits: el dividendo debe estar en AX, con el cociente en AL y el residuo en AH.
- Para divisores de 16 bits: el dividendo debe estar en DX:AX, con el cociente en AX y el residuo en DX.
- Para divisores de 32 bits: el dividendo debe estar en EDX:EAX, con el cociente en EAX y el residuo en EDX.

Instrucción div: La instrucción div realiza la división de números sin signo. El cociente y residuo se almacenan en registros según el tamaño de los operandos.

Ejemplo:

```
mov AX, 251
mov CL, 12
```

```
div CL ; Resultado: AL = 20, AH = 11
```

Aquí, 251 dividido por 12 da un cociente de 20 y un residuo de 11.

Instrucción idiv: La instrucción idiv realiza la división de números con signo. Una complicación adicional es que el dividendo debe extenderse con el signo. Esto se puede manejar con instrucciones como cbw, cwd, y cdq que extienden el signo del registro acumulador al siguiente registro de mayor tamaño.

Ejemplo:

```
mov AX, -251 ; AX = FF14H
mov CX, 12
cwd          ; Extiende el signo a DX
idiv CX      ; Resultado: AX = -21, DX = -11
```

IX. Instrucciones relacionadas

Para facilitar las operaciones con números firmados, las instrucciones **cbw**, **cwd**, y **cdq** se utilizan para extender el signo del acumulador a registros más grandes.

1. **cbw**: Convierte un byte (en AL) en un word (extiende el signo a AH).
2. **cwd**: Convierte un word (en AX) en un doubleword (extiende el signo a DX).
3. **cdq**: Convierte un doubleword (en EAX) en un quadword (extiende el signo a EDX).

Estas instrucciones son esenciales cuando se trabaja con divisiones de números con signo, ya que garantizan que el dividendo esté correctamente extendido con su signo.